Nested Checkbox Tree (Tri-state Parent)

- **Problem:** Build a nested checkbox tree. Checking a parent checks every child below it; checking some children puts the parent in a partially checked (indeterminate) state.
- **Tech stack:** React (Vite, JavaScript) (default, not specified)
- **Time limit:** 60 minutes (default, not specified)
- **Often asked at:** Google, Atlassian, Rippling

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements.** Restate the problem, split functional vs non-functional, ask clarifying questions, write down assumptions.
2. **List the interactions.** Every user action and its expected result. This is your checklist for building and testing.
3. **Design the architecture.** Name the components and their one-line jobs. Start small.
4. **Design the state.** What is the single source of truth? What can be derived?
5. **Design data structures and contracts.** Shape of the input data, props, return values.
6. **Trace one data flow end to end.** Proves the design works before you type.
7. **Find the hard parts.** Solve them on paper first.
8. **Build in layers.** Simplest working version first, then one working layer at a time.
9. **Cover states and edge cases.** Empty, error, boundaries.
10. **Wrap up.** Trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Interactions, architecture, state design	8 min
Hard parts on paper (status derivation, toggle rule)	5 min
Build: static recursive tree	10 min
Build: check / uncheck logic + indeterminate	15 min
Build: expand / collapse, select all, summary	10 min
Edge cases, accessibility, wrap-up	7 min

Tip: think out loud

Say what you are deciding and why: "I'll store only leaf ids, because parent state can always be computed from them, so it can never go out of sync." The interviewer grades your reasoning as much as the code.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Render a tree of any depth from nested data. Each node has a checkbox and a label.
- Checking a parent checks **all** its descendants. Unchecking a parent unchecks all descendants.
- Checking a child updates every ancestor: **checked** if all leaves below are checked, **unchecked** if none are, **indeterminate** if some are.
- Clicking an indeterminate parent checks everything below it.
- Expand / collapse parent nodes.
- Show which items are selected (count + list).
- Nice to have: "Select all", "Expand all", "Collapse all", "Clear", pre-selected values.

Non-functional requirements

Area	Requirement
Performance	One toggle must cost O(n) at most, no deep cloning of the tree on every click.
Scalability	Works for any depth and a few thousand nodes; virtualization later for very large trees.
Error handling	Bad input (unknown ids, empty folders, empty data) must not crash.
Accessibility	Native checkboxes (keyboard + screen reader for free), indeterminate announced as "mixed", expand buttons with `aria-expanded` and labels.
Reusability	`CheckboxTree` takes any `data` with `{ id, label, children }`; logic lives in pure functions and a hook.
Extensibility	Easy to add search/filter, disabled nodes, lazy-loaded children, controlled mode.

Clarifying questions to ask

1. What does the selection value contain: only **leaf** ids, or parent ids too? (Affects what we send to the backend.)
2. When I click an **indeterminate** parent, should it check all or uncheck all? (Most UIs, e.g. Gmail, check all.)
3. Can there be **empty folders** (parents with no children)? Are they selectable?
4. Is the tree depth bounded? How many nodes at most?

5. Are children loaded **lazily** from an API when a node expands, or is the whole tree given up front?
6. Do we need **disabled** nodes or search/filter in this round?

Assumptions we will build with

- The full tree is given up front as nested `{ id, label, children? }` with unique, stable string ids.
- The selection is the set of **checked leaf ids**. Parent state is derived.
- An empty folder (`children: []`) behaves like a leaf.
- Clicking an indeterminate parent checks all leaves below.
- Initially only top-level parents are expanded.
- Pre-selected ids may include a parent id (means "all its leaves") or unknown ids (ignored).

---

2. Identify interactions / use cases

Action / input	Expected behavior / output
Page loads with `initialChecked = ['pixel', 'laptops']`	Pixel, MacBook, ThinkPad checked. Phones and Electronics indeterminate, Laptops checked. "3 of 11 selected".
Check a leaf	Leaf checked; each ancestor recomputed (checked / indeterminate).
Uncheck the last checked leaf under a parent	Parent (and maybe ancestors) become unchecked.
Check an unchecked parent	All leaves below checked; parent checked; ancestors updated.
Click a checked parent	All leaves below unchecked.
Click an indeterminate parent	All leaves below checked.
Click "Select all"	All leaves checked (or all cleared if already all checked).
Click "Clear"	Nothing selected. Disabled when nothing is selected.
Click ▸ / ▾ on a parent	Children shown / hidden; checked state kept.
"Expand all" / "Collapse all"	Every parent opened / closed.
Check a parent while it is collapsed	Hidden children are checked too (state does not depend on what is rendered).
Press Space on a focused checkbox	Same as click (native checkbox).
`data = []`	"Nothing to select."

---

3. Design the architecture

Module	Responsibility
`data/categories.js`	Sample nested data (mock of an API response).
`utils/tree.js`	Pure tree functions: leaves, status map, toggle, normalize initial ids.

Module	Responsibility
`hooks/useCheckboxTree.js`	Owns state (`checked` , `expanded`), derives status, exposes actions.
`components/CheckboxTree.jsx`	Root UI: toolbar, tree list, selection summary. Calls the hook.
`components/TreeNode.jsx`	Recursive: renders one node and, if open, its children.
`components/Checkbox.jsx`	Checkbox that supports the tri-state (sets the `indeterminate` DOM property).

```
App
└── CheckboxTree            (useCheckboxTree → state + derived status)
    ├── Toolbar: Checkbox "Select all", Expand all, Collapse all, Clear
    ├── ul.tree
    │   └── TreeNode (recursive)
    │       ├── twisty button (expand/collapse)
    │       ├── Checkbox
    │       └── ul > TreeNode ... (children, if open)
    └── Summary + chips

utils/tree.js  ← used by the hook (pure, unit-testable)
```

Start with one component that renders a recursive list. Split out `Checkbox` once the indeterminate ref logic appears, and move logic to `utils` + a hook once it grows.

---

4. Design the state / data model

State or field	Purpose
`checked: Set<leafId>`	**Single source of truth** for selection. Only leaf ids.
`expanded: Set<parentId>`	Which parents are open. UI-only state.
`status: Map<id, 'checked' | 'unchecked' | 'indeterminate'>`	**Derived** with `useMemo` from `data` + `checked` . Never stored.
`root`	Derived virtual node `{ id: '__root__', children: data }` , so "Select all" is just another parent.
`leaves` , `selected`	Derived: all leaf nodes, and the checked ones in display order.

```
const [checked, setChecked] = useState(() => normalizeChecked(data, initialChecked));
const [expanded, setExpanded] = useState(() => new Set(data.filter((n) => !isLeaf(n)).map((n) =>
n.id)));

const root = useMemo(() => ({ id: ROOT_ID, label: 'Select all', children: data }), [data]);
const status = useMemo(() => buildStatusMap([root], checked), [root, checked]);
```

Senior note.

- *Derive, don't store.* Parent state is a pure function of its leaves. If we stored a `checked` flag on every node, each click would need to update children **and** walk up all ancestors, and any missed update leaves the tree inconsistent. Storing only leaves makes inconsistency impossible.
- *Don't mutate the data.* The tree from props stays read-only. We never copy/clone it on clicks; we only create a new `Set` of ids.
- *Ownership.* `checked` and `expanded` live in the hook used by `CheckboxTree`. `TreeNode` is stateless: it reads `status` and `expanded` and calls callbacks. Expanded state is separate from checked state, so collapsing never loses a selection.

---

5. Design the data structures and contracts

Example data

```
export const categories = [
  {
    id: 'electronics', label: 'Electronics',
    children: [
      { id: 'phones', label: 'Phones', children: [
        { id: 'iphone', label: 'iPhone' }, { id: 'pixel', label: 'Pixel' }, { id: 'galaxy', label:
'Galaxy' },
      ] },
      { id: 'laptops', label: 'Laptops', children: [
        { id: 'macbook', label: 'MacBook' }, { id: 'thinkpad', label: 'ThinkPad' },
      ] },
      { id: 'accessories', label: 'Accessories', children: [] }, // empty folder → leaf
    ],
  },
  { id: 'books', label: 'Books', children: [ /* fiction, non-fiction ... */ ] },
  { id: 'toys', label: 'Toys' }, // leaf at root level
];
```

Why stable ids? They are the React `key` (so React keeps the right DOM checkbox when the list changes) and the value we store in `checked`. Labels can repeat ("Other" under two parents); ids cannot. Never use array indexes.

Contracts

Component

```
<CheckboxTree
  data={TreeNode[]}          // { id: string, label: string, children?: TreeNode[] }
  initialChecked={string[]}  // optional; leaf or parent ids
/>
```

Pure functions (`utils/tree.js`)

Function	Input	Output
`isLeaf(node)`	node	`true` if no children or empty children
`getLeaves(node)`	node	leaf nodes below it, in display order
`getParentIds(nodes)`	nodes	ids of all non-leaf nodes
`buildStatusMap(nodes, checked)`	nodes, `Set`	`Map(id → status)`
`toggleNode(checked, node)`	`Set` , node	new `Set`
`normalizeChecked(nodes, ids)`	nodes, `string[]`	`Set` of leaf ids

Hook

```
const { root, status, expanded, leaves, selected,
        toggleCheck, clear, toggleExpand, expandAll, collapseAll } = useCheckboxTree(data,
initialChecked);
```

If it came from an API (mock): `GET /api/categories → 200 [{ id, label, children }]` , and the selection is saved as `PUT /api/preferences { categoryIds: ['pixel', 'macbook', 'thinkpad'] }` . Sending leaf ids keeps the payload unambiguous.

---

6. Trace the data flow

User clicks the **Phones** checkbox while Phones is indeterminate (only Pixel checked):

```
click on Phones checkbox
   ↓
Checkbox onChange → TreeNode → tree.toggleCheck(phonesNode)
   ↓
setChecked(prev => toggleNode(prev, phonesNode))
   ↓  toggleNode: leaf ids = [iphone, pixel, galaxy]
   ↓  not all checked → turnOn = true → new Set adds all three
   ↓
React re-renders CheckboxTree
   ↓
useMemo: buildStatusMap([root], checked)  (one post-order pass)
   ↓  phones: 3/3 → checked
   ↓  electronics: 5/6 → indeterminate (Accessories still off)
   ↓  root: 5/11 → indeterminate
   ↓
TreeNode reads status.get(id) → Checkbox
   ↓  checked = status === 'checked'
   ↓  useEffect sets input.indeterminate = status === 'indeterminate'
   ↓
UI: Phones ☑, Electronics ▣, Select all ▣, "5 of 11 selected"
```

---

7. Important technical challenges

Challenge 1: Keeping parents and children consistent

Problem. Store a `checked` flag on every node and you must sync in two directions: down (parent click → all descendants) and up (child click → every ancestor, which needs parent pointers). Scenario: check Pixel, then check Phones, then uncheck Electronics. Miss one ancestor update and Electronics shows checked while Galaxy is unchecked.

Solution. Store only checked **leaf ids** and derive every parent's state in one post-order pass. Each node returns `[checkedLeaves, totalLeaves]` to its parent. Cost O(n) per change; no parent pointers; impossible to go out of sync.

```
export function buildStatusMap(nodes, checked) {
  const status = new Map();
  const visit = (node) => {
    if (isLeaf(node)) {
      const on = checked.has(node.id);
      status.set(node.id, on ? CHECKED : UNCHECKED);
      return [on ? 1 : 0, 1];
    }
    let done = 0, total = 0;
    for (const child of node.children) {
      const [d, t] = visit(child);
      done += d; total += t;
    }
    status.set(node.id, done === 0 ? UNCHECKED : done === total ? CHECKED : INDETERMINATE);
    return [done, total];
  };
  nodes.forEach(visit);
  return status;
}
```

Challenge 2: The toggle rule for a parent (including indeterminate)

Problem. What does a click on a parent mean? Scenario: Phones is indeterminate (Pixel only). If we decided by "the parent's own checkbox was checked", we have no such flag. We need one rule that works for checked, unchecked and indeterminate.

Solution. Look at the leaves below the node: if **all** are checked, uncheck them all; otherwise check them all. Indeterminate therefore becomes checked. The same function works for leaves (a leaf is its own only leaf) and for "Select all" (a virtual root).

```
export function toggleNode(checked, node) {
  const ids = getLeaves(node).map((leaf) => leaf.id);
  const turnOn = !ids.every((id) => checked.has(id));
  const next = new Set(checked); // new Set → React sees a change
  for (const id of ids) {
    if (turnOn) next.add(id);
    else next.delete(id);
  }
  return next;
}
```

Use the functional updater `setChecked(prev => toggleNode(prev, node))` so fast double clicks always see the latest set.

Challenge 3: Showing the indeterminate state

Problem. HTML has no `indeterminate` attribute, and React has no `indeterminate` prop. It is a DOM **property** only. Also, when the user clicks an indeterminate checkbox, the browser clears `indeterminate` itself before React re-renders.

Solution. A small `Checkbox` component with a ref that writes the property after every render. `checked` stays controlled by React. Native checkboxes then expose "mixed" to screen readers automatically.

```
export default function Checkbox({ status, label, onChange }) {
  const ref = useRef(null);
  useEffect(() => {
    ref.current.indeterminate = status === INDETERMINATE;
  }); // no deps: re-apply after every render, the browser may have cleared it
  return (
    <label className="checkbox">
      <input ref={ref} type="checkbox" checked={status === CHECKED} onChange={onChange} />
      <span>{label}</span>
    </label>
  );
}
```

Challenge 4: Messy initial selection and empty folders

Problem. The backend sends `['pixel', 'laptops', 'old-id']`: a leaf, a parent and a deleted id. And `Accessories` has `children: []`, so "all children checked" is true for zero children (`[].every(...) === true`), which would show an empty folder as checked forever.

Solution. Normalize on init: a parent id expands to all its leaves, unknown ids are dropped. Treat `children: []` as a leaf (`isLeaf = !node.children?.length`), so it gets its own checkbox state.

```
export function normalizeChecked(nodes, ids) {
  const wanted = new Set(ids);
  const out = new Set();
  const visit = (node, inherited) => {
    const on = inherited || wanted.has(node.id);
    if (isLeaf(node)) { if (on) out.add(node.id); }
    else node.children.forEach((child) => visit(child, on));
  };
  nodes.forEach((node) => visit(node, false));
  return out;
}
```

---

8. Implement in the correct order

#	Step	Working result
1	Vite app, sample `categories.js`	Data ready
2	Recursive `TreeNode` rendering label + plain checkbox	Static tree on screen
3	`checked` Set of leaf ids + leaf toggle	Leaves can be checked
4	`buildStatusMap` (post-order) + use it for `checked` prop	Parents show checked when all leaves are
5	`Checkbox` with ref → indeterminate	Tri-state visible
6	`toggleNode` for parents	Parent click cascades down
7	Move logic into `utils/tree.js` + `useCheckboxTree`	Clean, testable
8	`expanded` Set + twisty buttons, Expand/Collapse all	Collapsible tree
9	Virtual root → "Select all", Clear	Toolbar works
10	Summary + chips of selected leaves	Visible output
11	`normalizeChecked`, empty folder, empty data	Edge cases safe
12	Accessibility pass (labels, `aria-expanded`, focus ring) + styles	Done

9. Cover all states and edge cases

State / case	Handling
Initial	Top-level parents expanded; `initialChecked` normalized into leaf ids.
Loading	Not needed (data is local). With an API: show "Loading…" until data arrives, then mount the tree (so `useState` init sees real data).
Success	Tree rendered; summary shows "N of M selected".
Empty data	"Nothing to select." No toolbar.
Error (API)	Show message + Retry button; do not render a half tree.
Nothing selected	All unchecked, "0 of M selected", Clear disabled.
All selected	Select all checked.
Partially selected	Ancestors indeterminate, Select all indeterminate.
Click indeterminate parent	Checks all leaves below.
Empty folder `children: []`	Treated as a leaf, has its own state.
Leaf at root level	Works like any leaf (Toys).
Initial ids include parent / unknown ids	Parent → all its leaves; unknown dropped.

State / case	Handling
Toggle while collapsed	Hidden children change too (state is not tied to the DOM).
Deep tree (10+ levels)	Recursion depth is the tree depth, fine for UI trees.
Duplicate labels	Safe, everything keys on ids.
Rapid clicks	Functional updater always uses latest state.
Keyboard	Tab to checkbox/button, Space toggles, Enter/Space on twisty.

10. Final structure

```
_60_nested_checkbox_tree/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── App.jsx
    ├── index.css
    ├── data/
    │   └── categories.js
    ├── utils/
    │   └── tree.js
    ├── hooks/
    │   └── useCheckboxTree.js
    └── components/
        ├── CheckboxTree.jsx
        ├── TreeNode.jsx
        └── Checkbox.jsx
```

11. Full working code

src/main.jsx

```
import { StrictMode } from 'react'
import { createRoot } from 'react-dom/client'
import './index.css'
import App from './App.jsx'

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>,
)
```

src/App.jsx

```
import CheckboxTree from './components/CheckboxTree.jsx';
import { categories } from './data/categories.js';

export default function App() {
  return (
    <main className="app">
      <h1>Nested Checkbox Tree</h1>
      {/* 'laptops' is a parent id: it pre-selects all its leaves */}
      <CheckboxTree data={categories} initialChecked={['pixel', 'laptops']} />
    </main>
  );
}
```

src/data/categories.js

```
// Stable string ids: used as React keys and as the values we store in `checked`
export const categories = [
  {
    id: 'electronics',
    label: 'Electronics',
    children: [
      {
        id: 'phones',
        label: 'Phones',
        children: [
          { id: 'iphone', label: 'iPhone' },
          { id: 'pixel', label: 'Pixel' },
          { id: 'galaxy', label: 'Galaxy' },
        ],
      },
      {
        id: 'laptops',
        label: 'Laptops',
        children: [
          { id: 'macbook', label: 'MacBook' },
          { id: 'thinkpad', label: 'ThinkPad' },
        ],
      },
      { id: 'accessories', label: 'Accessories', children: [] },
    ],
  },
  {
    id: 'books',
    label: 'Books',
    children: [
      {
        id: 'fiction',
        label: 'Fiction',
        children: [
          { id: 'novels', label: 'Novels' },
          { id: 'short-stories', label: 'Short stories' },
        ],
      },
      {
        id: 'non-fiction',
        label: 'Non-fiction',
        children: [
          { id: 'biography', label: 'Biography' },
          { id: 'history', label: 'History' },
        ],
```

```
      },
    ],
  },
  { id: 'toys', label: 'Toys' },
];
```

src/utils/tree.js

```
export const CHECKED = 'checked';
export const UNCHECKED = 'unchecked';
export const INDETERMINATE = 'indeterminate';

// A node with no children (or an empty folder) is selectable on its own
export const isLeaf = (node) => !node.children?.length;

// Leaf nodes under a node, in display order
export function getLeaves(node, out = []) {
  if (isLeaf(node)) out.push(node);
  else node.children.forEach((child) => getLeaves(child, out));
  return out;
}

export function getParentIds(nodes, out = []) {
  for (const node of nodes) {
    if (isLeaf(node)) continue;
    out.push(node.id);
    getParentIds(node.children, out);
  }
  return out;
}

// One post-order pass over the tree: O(n) per change.
// Returns Map(id -> CHECKED | UNCHECKED | INDETERMINATE)
export function buildStatusMap(nodes, checked) {
  const status = new Map();

  const visit = (node) => {
    if (isLeaf(node)) {
      const on = checked.has(node.id);
      status.set(node.id, on ? CHECKED : UNCHECKED);
      return [on ? 1 : 0, 1];
    }
    let done = 0;
    let total = 0;
    for (const child of node.children) {
      const [d, t] = visit(child);
      done += d;
      total += t;
    }
    status.set(node.id, done === 0 ? UNCHECKED : done === total ? CHECKED : INDETERMINATE);
    return [done, total];
  };
```

```
  nodes.forEach(visit);
  return status;
}

// Fully checked -> uncheck every leaf below; unchecked or indeterminate -> check every leaf
export function toggleNode(checked, node) {
  const ids = getLeaves(node).map((leaf) => leaf.id);
  const turnOn = !ids.every((id) => checked.has(id));
  const next = new Set(checked);
  for (const id of ids) {
    if (turnOn) next.add(id);
    else next.delete(id);
  }
  return next;
}

// Initial ids may name a parent (= all its leaves) or an unknown id (dropped)
export function normalizeChecked(nodes, ids) {
  const wanted = new Set(ids);
  const out = new Set();

  const visit = (node, inherited) => {
    const on = inherited || wanted.has(node.id);
    if (isLeaf(node)) {
      if (on) out.add(node.id);
    } else {
      node.children.forEach((child) => visit(child, on));
    }
  };

  nodes.forEach((node) => visit(node, false));
  return out;
}
```

src/hooks/useCheckboxTree.js

```
import { useMemo, useState } from 'react';
import { buildStatusMap, getLeaves, getParentIds, isLeaf, normalizeChecked, toggleNode } from
'../utils/tree.js';

const ROOT_ID = '__root__';

export function useCheckboxTree(data, initialChecked = []) {
  // Only leaf ids are stored; every parent state is derived from them
  const [checked, setChecked] = useState(() => normalizeChecked(data, initialChecked));
  const [expanded, setExpanded] = useState(() => new Set(data.filter((n) => !isLeaf(n)).map((n) =>
n.id)));

  // Virtual root makes "Select all" just another parent node
  const root = useMemo(() => ({ id: ROOT_ID, label: 'Select all', children: data }), [data]);
  const leaves = useMemo(() => getLeaves(root), [root]);
  const parentIds = useMemo(() => getParentIds(data), [data]);
  const status = useMemo(() => buildStatusMap([root], checked), [root, checked]);

  return {
    root,
    status,
    expanded,
    leaves,
    selected: leaves.filter((leaf) => checked.has(leaf.id)),
    toggleCheck: (node) => setChecked((prev) => toggleNode(prev, node)),
    clear: () => setChecked(new Set()),
    toggleExpand: (id) =>
      setExpanded((prev) => {
        const next = new Set(prev);
        if (next.has(id)) next.delete(id);
        else next.add(id);
        return next;
      }),
    expandAll: () => setExpanded(new Set(parentIds)),
    collapseAll: () => setExpanded(new Set()),
  };
}
```

src/components/Checkbox.jsx

```
import { useEffect, useRef } from 'react';
import { CHECKED, INDETERMINATE } from '../utils/tree.js';

export default function Checkbox({ status, label, onChange }) {
  const ref = useRef(null);

  // `indeterminate` is a DOM property only (no HTML attribute), and a native click clears it,
  // so re-apply it after every render
  useEffect(() => {
    ref.current.indeterminate = status === INDETERMINATE;
  });

  return (
    <label className="checkbox">
      <input ref={ref} type="checkbox" checked={status === CHECKED} onChange={onChange} />
      <span>{label}</span>
    </label>
  );
}
```

src/components/TreeNode.jsx

```
import Checkbox from './Checkbox.jsx';
import { isLeaf } from '../utils/tree.js';

export default function TreeNode({ node, status, expanded, onToggleCheck, onToggleExpand }) {
  const hasChildren = !isLeaf(node);
  const isOpen = expanded.has(node.id);

  return (
    <li className="tree-node">
      <div className="row">
        {hasChildren ? (
          <button
            type="button"
            className="twisty"
            aria-expanded={isOpen}
            aria-label={`${isOpen ? 'Collapse' : 'Expand'} ${node.label}`}
            onClick={() => onToggleExpand(node.id)}
          >
            {isOpen ? '▾' : '▸'}
          </button>
        ) : (
          <span className="twisty" aria-hidden="true" />
        )}
        <Checkbox status={status.get(node.id)} label={node.label} onChange={() => onToggleCheck(node)}
/>
      </div>

      {hasChildren && isOpen && (
        <ul className="tree-group">
          {node.children.map((child) => (
            <TreeNode
              key={child.id}
              node={child}
              status={status}
              expanded={expanded}
              onToggleCheck={onToggleCheck}
              onToggleExpand={onToggleExpand}
            />
          ))}
        </ul>
      )}
    </li>
  );
}
```

src/components/CheckboxTree.jsx

```
import Checkbox from './Checkbox.jsx';
import TreeNode from './TreeNode.jsx';
import { useCheckboxTree } from '../hooks/useCheckboxTree.js';

export default function CheckboxTree({ data, initialChecked }) {
  const tree = useCheckboxTree(data, initialChecked);

  if (!data.length) return <p className="empty">Nothing to select.</p>;

  return (
    <section className="checkbox-tree">
      <div className="toolbar">
        <Checkbox
          status={tree.status.get(tree.root.id)}
          label="Select all"
          onChange={() => tree.toggleCheck(tree.root)}
        />
        <div className="actions">
          <button type="button" onClick={tree.expandAll}>Expand all</button>
          <button type="button" onClick={tree.collapseAll}>Collapse all</button>
          <button type="button" onClick={tree.clear} disabled={!tree.selected.length}>Clear</button>
        </div>
      </div>

      <ul className="tree" aria-label="Categories">
        {data.map((node) => (
          <TreeNode
            key={node.id}
            node={node}
            status={tree.status}
            expanded={tree.expanded}
            onToggleCheck={tree.toggleCheck}
            onToggleExpand={tree.toggleExpand}
          />
        ))}
      </ul>

      <p className="summary" aria-live="polite">
        {tree.selected.length} of {tree.leaves.length} selected
      </p>
      {tree.selected.length > 0 && (
        <ul className="chips">
          {tree.selected.map((leaf) => (
            <li key={leaf.id}>{leaf.label}</li>
          ))}
```

```
        </ul>
      )}
    </section>
  );
}
```

src/index.css

```
* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  color: #1f2328;
  background: #f6f8fa;
}

.app { max-width: 520px; margin: 40px auto; padding: 0 16px; }

.checkbox-tree {
  background: #fff;
  border: 1px solid #d0d7de;
  border-radius: 8px;
  padding: 16px;
}

.toolbar {
  display: flex;
  flex-wrap: wrap;
  gap: 8px;
  justify-content: space-between;
  align-items: center;
  padding-bottom: 12px;
  border-bottom: 1px solid #d0d7de;
}

.actions { display: flex; gap: 6px; }

button {
  font: inherit;
  padding: 4px 10px;
  border: 1px solid #d0d7de;
  border-radius: 6px;
  background: #f6f8fa;
  cursor: pointer;
}
button:disabled { opacity: 0.5; cursor: not-allowed; }
button:focus-visible, input:focus-visible { outline: 2px solid #0969da; outline-offset: 2px; }

.tree, .tree-group { list-style: none; margin: 0; padding: 0; }
.tree { margin-top: 8px; }
.tree-group { padding-left: 22px; }
```

```
.row { display: flex; align-items: center; gap: 4px; padding: 2px 0; }

.twisty {
  width: 22px;
  height: 22px;
  padding: 0;
  border: none;
  background: none;
  flex-shrink: 0;
}

.checkbox { display: inline-flex; align-items: center; gap: 6px; cursor: pointer; }
.checkbox input { width: 16px; height: 16px; margin: 0; }

.summary { margin: 12px 0 6px; font-size: 14px; color: #57606a; }

.chips { display: flex; flex-wrap: wrap; gap: 6px; list-style: none; margin: 0; padding: 0; }
.chips li { padding: 2px 8px; background: #ddf4ff; border-radius: 999px; font-size: 13px; }

.empty { color: #57606a; }
```

---

12. Wrap-up

Edge cases handled

- Indeterminate parents at every level, including "Select all".
- Clicking indeterminate checks all; clicking checked unchecks all.
- Empty folders, leaves at root level, empty data.
- Initial ids that are parents or unknown.
- Toggling while collapsed; rapid clicks.
- Keyboard and screen reader support via native checkboxes.

Trade-offs to mention

- **Leaf-only state vs per-node flags.** Leaf-only is always consistent and simple; the cost is an O(n) pass per change, which is nothing below ~10k nodes.
- **Full recompute vs incremental update.** We recompute the whole status map per click. Incremental (walk only up from the changed node with parent pointers) is O(depth) but adds a parent map and more code.
- **Uncontrolled component.** `CheckboxTree` owns `checked`. For forms, lift it up: `checked` + `onChange` props (controlled) so the parent can save or reset.
- **Prop drilling** `status`, `expanded`, handlers through the recursion. Fine at this size; Context is an option if the node gets more props.
- **Native nested lists, not an ARIA** `tree` **widget.** Native checkboxes give keyboard and "mixed" for free. A full `role="tree"` needs roving tabindex and arrow-key handling.

What to add with more time

- Unit tests for `utils/tree.js` (Vitest) and RTL tests for click flows.
- Search/filter that keeps matching nodes and their ancestors, auto-expands them.
- Disabled nodes (skip them when cascading, and decide whether they count for the parent state).
- Lazy-loaded children on expand (status "unknown" until loaded).
- `React.memo` on `TreeNode` + virtualization (flatten visible nodes, render a window) for 10k+ nodes.
- Full ARIA tree keyboard model (arrows, Home/End, type-ahead).
- Controlled mode and an `onChange(selectedIds)` callback.

---

13. Counter questions the interviewer may ask

Requirements

Q1. Should the selection contain parent ids or only leaf ids?

Leaf ids are the unambiguous truth: "laptops" can be derived from MacBook + ThinkPad. If the backend wants compact data (e.g. "all of Electronics"), convert on submit: walk the status map and emit the highest fully checked node instead of its leaves.

Q2. Why does clicking an indeterminate parent check everything instead of unchecking?

It is the common convention (Gmail, file managers): the user usually wants "the rest too". Either way, decide it in one place: the `turnOn` line in `toggleNode`. Ask the interviewer and change one line if they prefer the opposite.

Q3. What if a parent has zero children?

`[].every(...)` is `true`, so a naive check shows it as permanently checked. We treat `children: []` as a leaf, so it has its own checkbox state and counts as one item.

State design

Q4. Why not store `checked` on every node in the tree?

Then a click must update all descendants and every ancestor, and every update needs an immutable copy of the path. One missed branch and the UI lies. With leaf ids only, parent state is computed, so it can never be wrong.

Q5. Why a `Set` and not an array or object?

`Set.has` is O(1) and add/delete are simple. We always create a **new** Set (`new Set(prev)`), because React compares by reference; mutating the same Set would not re-render.

Q6. Why is `expanded` separate from `checked`?

They change for different reasons. Collapsing a node must not touch the selection, and "Clear" must not collapse anything. Separate state keeps both simple.

Q7. Why `useMemo` for the status map?

It only changes when `data` or `checked` changes. Without memo it would still be correct, just recomputed on unrelated renders (e.g. expand/collapse). It is cheap insurance, not a correctness need.

Hard parts

Q8. Why not use `useEffect` to update parents after a child changes?

That is a sync-state-with-state pattern: extra render, flicker risk and loops. Derived values belong in render (`useMemo`), not in effects.

Q9. Why does the indeterminate effect have no dependency array?

When the user clicks an indeterminate box, the browser sets `indeterminate = false` itself. If the status did not change (e.g. future disabled leaves), an effect with `[status]` would not re-run and the DOM would be wrong. Writing one boolean per render is cheap.

Q10. How does "Select all" work without extra code?

It is a virtual root node `{ id: '__root__', children: data }`. `buildStatusMap([root])` gives its status and `toggleNode(root)` toggles every leaf. Same code paths as any parent.

Q11. Why the functional updater `setChecked(prev => ...)`?

Two quick clicks can be processed before a re-render. Using `prev` guarantees each toggle starts from the latest state instead of a stale closure.

Performance and scale

Q12. What if the tree has 100,000 nodes?

Rendering is the bottleneck, not the logic. Flatten visible nodes into an array and virtualize (render ~50 rows). For logic, keep a parent map and update only the changed node's ancestors (O(depth)) plus its subtree, instead of the full O(n) pass. Memoize `TreeNode` so unchanged rows skip rendering.

Q13. Every click re-renders the whole tree. Is that a problem?

At a few hundred nodes, no. To optimize: `React.memo(TreeNode)` and pass each node its own primitive status (`status.get(id)`) instead of the whole Map, plus stable callbacks with `useCallback`. Then only nodes whose status changed re-render.

Q14. What if children are loaded from an API when expanding?

Store children in a `Map(parentId → children)` filled on first expand, show a spinner row while loading, and handle errors with retry. A checked parent whose children are not loaded yet needs a rule: either store the parent id as "fully selected" and expand it to leaves after loading, or load before allowing the check.

Trade-offs and "why not X?"

Q15. Why recursion and not an iterative stack?

The UI tree is a recursive structure and recursive components read naturally. Depth is small in practice. For very deep data (thousands of levels) an explicit stack avoids call-stack limits, but no real UI is that deep.

Q16. Why not use a library like `react-checkbox-tree`?

In an interview the point is to show the logic. In production a library is fine if it supports our needs (a11y, virtualization); otherwise this is ~150 lines we fully control.

Q17. How would you add search?

Filter to nodes whose label matches **plus their ancestors**, auto-expand those ancestors, and decide what a parent click means while filtered (usually: only visible leaves). Keep `checked` untouched by the filter so clearing the search restores everything.

Q18. How would you support disabled nodes?

Skip disabled leaves in `toggleNode` (they keep their value) and decide if they count toward parent status. A common rule: a parent is "checked" when all **enabled** leaves are checked.

Q19. How would the parent component get the selected values?

Make it controlled: `<CheckboxTree data checked={ids} onChange={setIds} />`. The hook then reads `checked` from props and calls `onChange(toggleNode(checked, node))` instead of `setChecked`. No effect needed.

Q20. How would you test this?

Unit-test the pure functions: `normalizeChecked` (parent id, unknown id), `buildStatusMap` (all / some / none, empty folder), `toggleNode` (checked → off, indeterminate → on). Then RTL: click Pixel, assert Phones `indeterminate === true`; click Electronics, assert all its leaves are checked; Space key on a checkbox.